본 자료는 저작권법 제25조 2항에 의해 보호를 받으며, **pdf 형태로 보실 수 있도록 공유드리고 있습니다.**
본 자료를 인터넷상에 오픈 또는 공유하시면, 쉽게 확인이 되어 저작권법에 의해 부득이 조치를 취할 수 밖에 없으므로, 인터넷상에 오픈 또는 공유는 하지 말아주세요
추가로 **잔재미코딩 가장 빠른 풀스택 로드맵**과 함께 익히시면, 보다 자유자재로 바이브 코딩을 익히고 활용할 수 있습니다
관련 강의도 **잔재미코딩(노이먼스랩)** 에서 확인하실 수 있으며,
해당 사이트에서 회원가입하시면, 최신 AI 및 자동화, 데이터, 풀스택 관련 트렌드 뉴스레터도 공유받으실 수 있습니다.

Claude Code 빠른 시작 가이드

1. 시작하기

Claude Code 실행

```
# 프로젝트 폴더로 이동
cd ~/Projects/my-project

# Claude Code 시작
claude
```

> **Windows 사용자 (Cursor/VS Code 터미널):** 터미널이 기본적으로 PowerShell로 열리며, 처음 실행 시 "스크립트를 실행할 수 없습니다" 오류가 발생할 수 있습니다. 이 경우 아래 명령어를 복사하여 붙여넣고 실행하세요:
>
> ```
> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
> ```
>
> "예(Y)"를 입력하면 이후 `claude` 명령어가 정상 작동합니다.

줄바꿈 설정

- **모든 OS**: 백슬래시(`\`) + Enter
- **macOS iTerm2**: `/terminal-setup` 실행 후 Shift + Enter 사용 가능

모델 선택

```
> /model
```

- **Sonnet 4.5** (기본 권장): 일반적인 개발 작업
- **Opus 4.1**: 복잡한 추론이 필요한 작업

---

2. 필수 단축키

단축키	기능	설명
`/`	슬래시 명령어	`/help` , `/init` , `/clear` 등
`@`	파일 참조	`@src/app.ts 설명해줘`
`Shift+Enter`	새 줄	멀티라인 프롬프트
`Shift+Tab`	Plan Mode 전환	계획 모드 켜기/끄기
`Tab`	Extended Thinking	심층 분석 켜기/끄기
`Esc+Esc`	되돌리기/입력 초기화	Rewind 또는 현재 입력 삭제

---

3. 핵심 기능

@ 기호로 파일 참조

프롬프트에서 `@` 를 입력하면 프로젝트 파일 목록이 자동완성으로 표시됩니다.

```
# 특정 파일 분석
> @package.json 파일의 의존성을 설명해주세요

# 파일 수정 요청
> @src/App.tsx의 버튼 색상을 파란색으로 변경해주세요

# 여러 파일 비교

> @package.json과 @README.md를 비교해서 문서가 최신인지 확인해주세요

# 폴더 전체 참조
> @src/components 폴더의 모든 파일을 검토해줘

# 부분 검색으로 빠르게 찾기
> @pack          # package.json을 찾음
> @App           # App.tsx, App.js 등을 찾음
```

팁:

- 파일 경로 전체를 입력할 필요 없음
- Tab 키나 방향키로 목록 탐색
- @ 로 참조한 파일은 자동으로 컨텍스트에 포함됨

이미지 붙여넣기

Claude Code는 이미지를 직접 분석할 수 있습니다.

사용 방법:

1. 이미지 파일을 클립보드에 복사 (Cmd+C / Ctrl+C)
2. Claude Code 프롬프트에서 붙여넣기 (Cmd+V / Ctrl+V)
3. 이미지와 함께 질문 입력

```
# 스크린샷 분석
> (이미지 붙여넣기) 이 에러 메시지를 분석하고 해결 방법을 알려줘
```

```
# 디자인 구현
> (목업 이미지 붙여넣기) 이 디자인대로 React 컴포넌트를 만들어줘

# UI 버그 리포트
> (버그 스크린샷 붙여넣기) 이 UI 문제의 원인을 찾아줘
```

지원 형식: PNG, JPG, JPEG, GIF, WebP

세션 관리

세션 종료:

```
/quit          # 또는 /exit
```

이전 세션 이어가기:

```
claude -r      # 세션 목록 보고 선택
claude -c      # 가장 최근 세션 바로 이어가기
```

---

4. 가장 자주 쓰는 슬래시 명령어

프로젝트 시작

```
/init              # 프로젝트 초기화 (CLAUDE.md 자동 생성)
/model             # 모델 선택 (Sonnet 4.5 / Opus 4.1)
```

대화 관리

```
/clear              # 대화 완전 삭제 (새 작업 시작)
/compact            # 대화 압축 (정보 유지하며 정리)
/context            # 컨텍스트 사용량 확인
```

작업 관리

```
/todos              # TODO 목록 확인 (Ctrl+T로도 가능)
/rewind             # 이전 상태로 되돌리기
/memory             # CLAUDE.md 파일 편집
```

IDE 연동

```
/ide                # Cursor AI, VS Code 등 IDE 열기
```

시스템 확인

```
/usage              # 플랜 사용량 확인
/doctor             # 설치 상태 진단
/mcp                # MCP 서버 상태 확인
/help               # 도움말 보기
```

---

5. CLAUDE.md: 프로젝트 영구 메모리

CLAUDE.md는 Claude Code의 영구 메모리로, 프로젝트 루트에 작성하면 모든 세션에서 자동 로드됩니다.

기본 템플릿 (100줄 이내 권장)

```
# 프로젝트: TODO 앱

## 목적
```

```
## 목적
개인용 할 일 관리 웹앱

## 기술 스택
- Backend: FastAPI
- Frontend: HTML, CSS, JavaScript

## 핵심 규칙
- 변수명은 명확한 영어로
- API 응답은 JSON 형식

## 프로젝트 구조
todo-app/
├── backend/      # FastAPI 기반 백엔드
├── frontend/     # HTML,CSS,JS 기반 프론트엔드
└── designs/      # 참고 디자인 파일
```

활용 예시

```
> /init                # CLAUDE.md 자동 생성
> /memory              # CLAUDE.md 편집
```

📖 **상세 가이드:** 별도 문서 `1001_CLAUDECODE_QUICKPROJECTSTART.md` 참조

---

6. XML로 화면 디자인하기

말로 설명하면 AI가 색상·크기·배치를 정확히 이해하기 어렵습니다. XML로 작성하면 명확합니다.

간단한 예시

```
<화면 이름="TODO앱">
  <제목 색상="#333" 크기="32px">할 일 목록</제목>

  <입력영역>
    <입력창 너비="70%" 힌트="할 일 입력" />
```

```
    <입력창 너비="70%" 힌트="할 일 입력" />
    <버튼 색상="#007bff">추가</버튼>
  </입력영역>

  <할일목록>
    <할일항목>
      <체크박스 />
      <내용>장보기</내용>
      <삭제버튼 색상="#dc3545">삭제</삭제버튼>
    </할일항목>
  </할일목록>
</화면>
```

AI에게 적용 요청

```
> @designs/main-screen.xml 참고해서 HTML/CSS 만들어줘
```

💡 **팁:** AI에게 XML 초안 작성 맡겨도 OK!

```
> TODO 앱 메인 화면을 XML로 디자인해줘. designs/main.xml로 저장
```

---

7. 실행 모드

Interactive Mode (기본)

```
claude                                  # 일반 모드
claude --dangerously-skip-permissions   # YOLO 모드
```

- 일반적인 개발 작업에 사용
- 대화형으로 지속적인 작업 가능

Plan Mode (복잡한 작업)

Plan Mode = 실제 파일 수정 없이 계획만 세우고 검토하기

시작 방법:

- 명령줄: `claude --permission-mode plan`
- Interactive에서: **Shift+Tab**으로 전환

특징:

- 실행 전 계획을 먼저 수립
- 여러 파일 수정이 필요한 복잡한 작업에 유용
- TodoList와 작업 계획을 먼저 생성

모드 전환 (Shift+Tab):

- 일반 모드: ▸▸ accept edits ↔ ⏸ plan mode
- YOLO 모드: ▸▸ bypass permissions ↔ ▸▸ accept edits ↔ ⏸ plan mode

실전 사용 예시:

```
# 1. Plan Mode로 시작 (Shift+Tab)
[Shift + Tab]
⏸ plan mode on

> 다국어 지원 기능 추가해줘

# Claude가 계획과 TodoList 생성:
# [ ] translations.js 작성
# [ ] i18n.js 로직 구현
# [ ] 5개 HTML 페이지 수정

# [ ] 테스트 및 검증

# 2. 계획 검토 후 결정
# 👍 좋으면 → Plan Mode 해제하고 실행
[Shift + Tab]
```

```
✓ plan mode off
> 계획대로 진행해줘

# 👎 수정 필요하면 → Plan Mode 유지하고 다시 요청
> HTML 페이지는 3개만 수정하자

# 명령줄로 바로 시작:
claude --permission-mode plan "기능 추가 계획 세워줘"
```

TodoList 상태

- `[ ]` pending (대기 중)
- `[~]` in_progress (진행 중)
- `[✓]` completed (완료)

병렬 실행 팁

독립적인 작업은 "병렬로" 또는 "동시에"라고 명시적으로 요청하면 훨씬 빠릅니다.

```
> 5개 HTML 파일을 병렬로 수정해줘  # 3배 빠름
```

병렬 실행 키워드:

```
> ... 병렬로 ...
> ... 동시에 ...
> ... parallel ...
> ... 그룹별로 ...
> ... 배치로 ...
```

Command Mode (일회성 실행)

```
# 기본 사용
claude -p "이 프로젝트의 주요 기능을 설명해주세요"

# 파이프 활용
cat error.log | claude -p "이 에러 로그를 분석해주세요"

# Plan Mode와 결합
claude --permission-mode plan -p "사용자 인증 시스템 구현 계획 세워줘"
```

---

8. Extended Thinking

활성화 방법

- **Tab 키**: 전체 세션에 적용 (켜기/끄기 토글)
- **키워드**: 특정 프롬프트에만 적용 (Tab OFF에서도 작동)

추론 수준

```
# 기본 추론 (4,000 토큰)
> think: 이 함수에 에러 핸들링 추가해줘

# 중간 추론 (10,000 토큰)
> 이 인증 플로우의 보안 취약점을 분석해줘, think hard

# 최대 추론 (31,999 토큰)
> 마이크로서비스 전환 계획을 수립해줘 ultrathink
```

키워드 정리

- **think**: `think`
- **think hard**: `think hard` , `think deeply` , `think about it`
- **ultrathink**: `ultrathink` , `think harder` , `think really hard`

입문자 황금 패턴

1. 🚀 **프로젝트 시작**: ultrathink (한 번만 제대로)
2. ⚡ **대부분의 구현**: 키워드 없이 (빠르게 진행)
3. 🔥 **막혔을 때**: think hard 또는 ultrathink (선택적 사용)

---

9. 컨텍스트 관리 핵심

컨텍스트란?

Claude가 한 번에 기억할 수 있는 최대 정보량 (Sonnet 4.5: 200,000 토큰)

컨텍스트 확인

```
/context            # 현재 사용량 시각화

# 출력 예시:
# Context Usage 164k/200k tokens (82%)
# ⛁⛁⛁⛁⛁⛁⛁⛶⛶⛶
# ⛶⛶⛶⛶⛶⛶⛶⛶⛶⛶
```

컨텍스트 구성 (예시)

```
200,000 토큰
├── 시스템 프롬프트: 2,800 (1.4%)
├── 시스템 도구: 11,500 (5.8%)
├── 예약 공간: 45,000 (22.5%)
├── MCP 도구: 15,400 (7.7%)
├── CLAUDE.md: 500 (0.3%)
└── 대화 메시지: 88,700 (44.3%) ← 가장 큰 비중
```

컨텍스트 관리 전략

1. /compact vs /clear

명령어	동작	사용 시점
/compact	대화 요약 보존	같은 작업 계속 진행
/clear	대화 완전 삭제	완전히 새 작업 시작

2. /compact 최적 타이밍

```
# ✅ 좋은 타이밍
- 기능 완성 후
- Git 커밋 직후
- 작업 전환 전 (프론트엔드 → 백엔드)
- 컨텍스트 80% 도달 시

# 예시
> 인증 기능 구현 완료!
> git commit -m "Add JWT authentication"
> /compact 인증 완료. 다음은 프로필 기능 작업 예정
```

자동 압축 이해

- 컨텍스트가 **95% 이상** 차면 자동으로 압축 실행
- 예측 불가능하므로 **80%에서 수동으로 /compact** 권장

---

10. YOLO 모드 (입문자 권장)

YOLO 모드란?

모든 권한을 자동 승인하여 학습 흐름을 끊지 않고 빠르게 체험할 수 있는 모드

시작하기

```
# YOLO 모드 시작
claude --dangerously-skip-permissions

# 일회성 작업
claude -p "README.md 파일을 만들어줘" --dangerously-skip-permissions
```

안전장치: /rewind

```
> /rewind
```

변경사항을 체크포인트 목록에서 선택하여 되돌릴 수 있습니다.

제한사항:

- Bash 명령(`rm` , `mv` , `cp`)으로 수정한 파일은 복구 불가
- Claude Code 외부에서 수정한 파일은 복구 불가

안전한 사용법

1. **테스트 프로젝트**에서 먼저 시작
2. **작은 작업**부터 진행
3. 중요한 프로젝트는 **Git 사용** 권장

---

11. 실전 워크플로우

새 프로젝트 시작

```
mkdir my-project
cd my-project
claude --dangerously-skip-permissions

> 블로그 플랫폼을 만들어줘. Markdown 에디터, 태그 기능 포함. ultrathink
```

기존 프로젝트 분석

```
claude

> 이 프로젝트의 전체 구조를 설명해줘
> @package.json 의존성을 검토해줘
> 폴더 구조를 설명해줘
```

복잡한 기능 구현

```
claude

# 1. Plan Mode로 계획 수립
> (Shift+Tab으로 Plan Mode 진입)
> 사용자 인증 시스템을 JWT 기반으로 구현해줘

# 2. 계획 확인 후 실행
> 계획이 좋아. 진행해줘

# 3. Plan Mode 종료
> (Shift+Tab)
> yes
```

버그 디버깅

```
> 로그인 시 500 에러가 발생해. 원인을 think hard 찾아줘
> @auth.js 파일을 확인해봐
```

```
> 데이터베이스 연결 부분을 자세히 살펴봐줘
```

---

12. /ide 명령: IDE 연동하기

Claude Code를 Cursor AI 같은 다른 IDE와 함께 사용할 수 있습니다.

기본 사용:

```
> /ide
```

실행하면 IDE 선택 화면이 나타납니다:

```
Which IDE would you like to open?
❯ 1. Cursor
  2. VS Code
  3. Other
```

특징:

- Claude Code에서 작업하면서 IDE에서 실시간 확인
- 양방향 워크플로우: Claude Code로 생성 → IDE에서 미세 조정
- 파일 동기화 자동 유지

활용 예시:

```
# Claude Code에서 작업 시작
> 새로운 React 컴포넌트를 만들어줘

# IDE에서 확인하고 싶을 때
> /ide

# Cursor 선택 → 자동으로 Cursor에서 프로젝트 열림
```

13. 핵심 팁

효율적인 프롬프트

- 구체적으로 요청하기
- @ 기호로 파일 명시하기
- 여러 조건은 줄바꿈으로 구분하기

```
# 좋은 예시
> @src/auth.js 파일의 로그인 함수를 수정해줘: \
  1. JWT 토큰 만료 시간을 24시간으로 설정 \
  2. 에러 메시지를 더 구체적으로 변경 \
  3. 로깅 추가
```

토큰 절약

- 기본은 Extended Thinking OFF
- 복잡한 작업만 키워드 사용
- 필요 시 `/compact` 로 컨텍스트 압축
- CLAUDE.md는 100줄 이내로 간결하게

안전한 작업

- 중요한 프로젝트는 Git 사용
- 복잡한 작업은 Plan Mode 활용
- 문제 발생 시 즉시 /rewind
- 정기적으로 `/context` 로 사용량 확인

14. 빠른 참조

작업	명령

Claude Code 시작	`claude`
YOLO 모드 시작	`claude --dangerously-skip-permissions`
파일 참조	`@파일명`
이미지 붙이기	`Cmd+V` / `Ctrl+V 또는 Alt+V` (클립보드에서)
IDE 연동	`/ide`
Plan Mode 전환	`Shift+Tab`
Extended Thinking	`Tab` (켜기/끄기)
되돌리기	`/rewind`
심층 분석	`ultrathink` 키워드
컨텍스트 확인	`/context`
대화 압축	`/compact`
대화 초기화	`/clear`
세션 종료	`/quit`
이전 세션	`claude -r`

본 자료는 저작권법 제25조 2항에 의해 보호를 받으며, **pdf 형태로 보실 수 있도록 공유드리고 있습니다.**

본 자료를 인터넷상에 오픈 또는 공유하시면, 쉽게 확인이 되어 저작권법에 의해 부득이 조치를 취할 수 밖에 없으므로, 인터넷상에 오픈 또는 공유는 하지 말아주세요
추가로 **잔재미코딩 가장 빠른 풀스택 로드맵**과 함께 익히시면, 보다 자유자재로 바이브 코딩을 익히고 활용할 수 있습니다
관련 강의도 **잔재미코딩(노이먼스랩)** 에서 확인하실 수 있으며,
해당 사이트에서 회원가입하시면, 최신 AI 및 자동화, 데이터, 풀스택 관련 트렌드 뉴스레터도 공유받으실 수 있습니다.